

Lustre Find Utility (LFU) Requirements

Project Objective

Lustre Find Utility (LFU) is a server-side namespace scanning and reporting subsystem targeted for Lustre 2.19+ (TLC). It replaces and unifies the ad-hoc scanners currently used across Lustre (“Lester the Lustre Lister”, OSD OI Scrub, Changelog crawl, client-side `lfs find`) with a structured, parallel, modular Object Stream pipeline running directly on MDTs and OSTs. LFU aims to deliver fast, scalable, attribute-filtered file discovery to downstream consumers (FLR mirror/EC resync, PCC-RO cache, Trash Can cleanup, HSM, tiered storage, third-party tools) with minimal data transfer, no intermediate database, and a clean open API for third-party integration.

References and Important Links

Link	Description	Notes
LUG 2026: Lustre 2.18 and Beyond (Dilger)	Slides 19–22: LFU overview, data flow, consumers and implementation roadmap	Primary design reference
<i>TLU ticket — to be added</i>	LFU tracking ticket in Jira	

Key Use Cases

- **HPC namespace management:** Administrators need to identify files untouched for 30+ days consuming more than 1 GB to archive to HSM — today this requires slow client-side `lfs find` traversals that scale poorly to billion-file filesystems.
- **Cloud and Hyperscaler operators:** Identify stale FLR mirrors needing resync without launching a full client-side directory crawl across all MDTs.
- **Tiered storage and PCC-RO:** Automatically promote or demote files based on atime, size, and project ID attributes using a server-side scan that feeds the tiered storage manager and PCC-RO cache.
- **HSM integration (S3, GCS):** Feed the HSM backend with a continuous, filtered Object Stream of files matching archival criteria without a separate scanner process.

- **Third-party tools:** RobinHood, PoliMor, and GUFi currently perform independent full-namespace scans. An open LFU scanner API lets these tools reuse the optimized server-side pipeline.
- **Admin reporting:** Generate on-demand reports on largest files, most-recently-accessed files, or files with stale mirrors using MDT-side indexes for near-instant results.

Current Lustre Limitations

Priority	Problem	Description	Desired Improvement	Related Tickets	TLU Ticket
Highest	No efficient server-side namespace scanner	<code>lfs find</code> crawls the directory tree from the client, transferring full MDT inode data just to filter it on the client side. This is extremely slow for large namespaces (>1B files).	Server-based scanner reads inodes directly at the OSD layer and sends only matching/requested attributes to the client, dramatically reducing network traffic and scan time.		
Highest	No unified scan pipeline	Multiple overlapping scanners exist (Lester,	A single modular pipeline with pluggable scanner		

		OSD OI Scrub, Changelog crawler, lfs find) with incompatible interfaces, duplicated logic, and no way to share a single scan across consumers.	backends, a common Object Stream format, and a Merge operator so one scan can feed multiple downstream consumers simultaneously.		
High	No structured binary Object Stream	Current tools emit text or ad-hoc binary formats, making downstream processing expensive, fragile, and difficult to evolve without breaking	Adopt a structured binary format (FlatBuffers or MsgPack) for the Object Stream, providing efficient serialization, schema evolution, and language-neutral		

		consumer s.	consumpti on.		
High	No MDT- side indexes for common query patterns	Queries on atime, largest files, or stale mirrors require a full namespac e scan regardless of result- set size, wasting I/O and time.	Add specific MDT indexes (atime, file size, mirror status) that enable O(result- set) scans instead of O(namesp ace) scans for common query patterns.		
High	No parallel multi-MDT scanning with stream merge	Scanning across multiple MDTs is sequential today; results from different MDTs cannot be merged or sorted server- side.	Parallel per-MDT scanners with a Merge Object Stream operator that combines and optionally sorts streams before		

			delivering results to userspace or a consumer module.		
Medium	No zero-copy kernel→userspace export	Scan results must be copied through multiple kernel/userspace boundaries, adding latency and CPU overhead for high-throughput scans.	Zero-copy lockless ring-buffer export from the kernel scanner to userspace consumers, eliminating unnecessary copies on the hot path.		
Medium	No access-control filters on server-side scan RPCs	A server-side scan could expose inode metadata for all files to any caller, violating per-user and per-project access	Access control filters on the remote bulk RPC transport so the server enforces user/project permissions before returning		

		boundaries.	Object Stream entries.		
Medium	Consumers re-implement scanning logic	PCC-RO cache manager, FLR mirror/EC resync, HSM, and tiered storage each implement their own namespace scan, leading to duplicated code, inconsistent behavior, and N independent full scans when multiple features are active.	A single LFU API that all Lustre subsystems consume, so one server-side scan can satisfy multiple feature consumers concurrently.		
Low	Third-party tools scan inefficiently	RobinHood, PoliMor, and GUFFI perform independent	Expose a documented, open scanner API so		

		nt full namespac e scans using client-side directory traversal, consumin g significant client and network resources.	external tools can plug into LFU and reuse the optimized server- side Object Stream pipeline instead of running their own crawls.	
--	--	--	---	--

Desired User Experience

1. `lfs find` backed by LFU returns results in seconds on billion-file filesystems by scanning inodes server-side and sending only matching entries to the client.
2. A single `lfs find /lfs02 -atime +30d -blocks +1G -projid 1999` invocation triggers one server-side scan that simultaneously feeds the FLR resync queue, the PCC-RO eviction list, and HSM archival — no repeated crawls.
3. MDT-side atime and size indexes make common queries (largest files, coldest data) near-instantaneous regardless of namespace size.
4. Third-party tools (RobinHood, PoliMor, GUFU) plug into LFU via an open API and receive filtered Object Streams without launching their own directory traversals.
5. The system can be rolled out incrementally: server-only mode first (improves `lfs find`), then extended to PtIRPC transport and full user access, with no flag day.

High-Level Features and Improvements

🔗 [TLU-186: Lustre Find Utility \(LFU\)](#) In Progress